# Can Data Beat the Market? Predicting NBA Games and Evaluating Kalshi Odds — Exploratory Data Analysis

Author: Winston Wihono
GitHub repository: https://github.com/wwihono/kalshi-reader

## Summary of Research Questions

1. How accurately do Kalshi's pregame NBA prices represent the actual probability of each team winning? I will compare market-implied probabilities shortly before tip-off with realized game results (calibration by probability bins).

2. Can an NBA prediction model outperform Kalshi's market probabilities? I will train logistic regression, random forest, and gradient boosting models on pregame features and compare accuracy, Brier score, and log loss on a chronological test set.

3. Under what conditions are Kalshi's NBA predictions most and least accurate? I will condition market accuracy and Brier score on volume, probability range, rest differences, and recent form gaps.

4. Would differences between the model and Kalshi's prices produce positive simulated returns? When model – Kalshi exceeds 5%, 10%, or 15%, I will simulate buying one contract and report historical ROI (no real-money trading).

## Motivation

Prediction markets such as Kalshi let traders buy and sell contracts whose prices can be read as approximate event probabilities. Those prices reflect information, liquidity, and opinion—not a guaranteed true probability. NBA games are a strong test case because teams play often and leave measurable pregame signals (form, rest, home court, and score differentials). This project evaluates whether Kalshi NBA prices are calibrated, when they err, and whether models that disagree with the market retain an edge on unseen games. It does not assume profitable trading is possible; it tests that claim carefully with held-out chronological evaluation.

## Dataset

Kalshi NBA Market Data. Public API markets for series ticker KXNBAGAME are downloaded from the recent markets endpoint and the historical markets endpoint (selected using Kalshi's historical cutoff). Important fields include ticker, event_ticker, title, yes_sub_title, occurrence_datetime / expected_expiration_time, volume_fp, result, and settlement_value_dollars. Candlestick endpoints supply pregame bid/ask history for later modeling; this EDA focuses on market structure, volume, settlement, and join quality with results.

NBA Schedule and Results Data. Basketball Reference's 2025–26 schedule pages provide game_date, visitor_team, home_team, visitor_pts, and home_pts. Monthly HTML tables are scraped and concatenated in Python, then de-duplicated because landing/month pages can repeat early-season games.

How the datasets relate: Kalshi KXNBAGAME contracts and Basketball Reference box scores describe the same NBA games. They are linked on game_date + home_team + visitor_team after team-name normalization (and team-pair matching for Kalshi 'vs' titles).

## Method

EDA steps: (1) paginate and save raw Kalshi JSON plus the Basketball Reference schedule CSV; (2) parse Kalshi titles/tickers into game dates and team labels, mapping short city names and ticker suffixes to canonical NBA names; (3) collapse the two Yes-contracts per game into game-level rows; (4) inner-join to Basketball Reference on date + home/visitor (using unordered team-pair matching when Kalshi titles use "vs"); (5) compute missingness, seven-number summaries, and categorical counts for variables tied to the research questions; (6) produce at least two visualizations for each analysis table (source tables and the joined table). All steps are implemented as runnable Python modules under src/kalshi_nba with scripts/run_eda.py as the entrypoint.

## EDA Results

## Kalshi markets (contract-level)

The Kalshi table has 2898 rows and 54 columns after de-duplicating tickers across recent/historical pulls. Rows represent One Yes-contract market for a team in an NBA game. Columns represent Market metadata, prices, volume, settlement, parsed teams/dates. There are 1449 distinct event_ticker values (games).

Missing data is present (confirmed with DataFrame.isna().any().any()). Highest-missing fields include: occurrence_datetime=2734 (94.3%); home_team=1328 (45.8%); visitor_team=1328 (45.8%); team_a=2 (0.1%); yes_team=1 (0.0%). Plan: prefer ticker-encoded game dates and expected_expiration_time when occurrence_datetime is absent; drop non-NBA exhibition contracts that fail team mapping; require complete join keys before modeling; obtain pregame probabilities from candlesticks rather than post-settlement yes_bid/yes_ask (which collapse to 0/1).

Variables of interest: volume_fp (liquidity / participation for RQ3), last_price_dollars and settlement_value_dollars / result (outcome labels for RQ1), status and title_format (data-quality diagnostics for messy-data challenge). These columns directly support calibration, conditioning on volume, and merge QA.

```
volume_fp: mean=4.029e+06, std=5.758e+06, min=1.944e+04, Q1=1.375e+06,
median=2.678e+06, Q3=4.582e+06, max=1.118e+08 (n=2898)
```

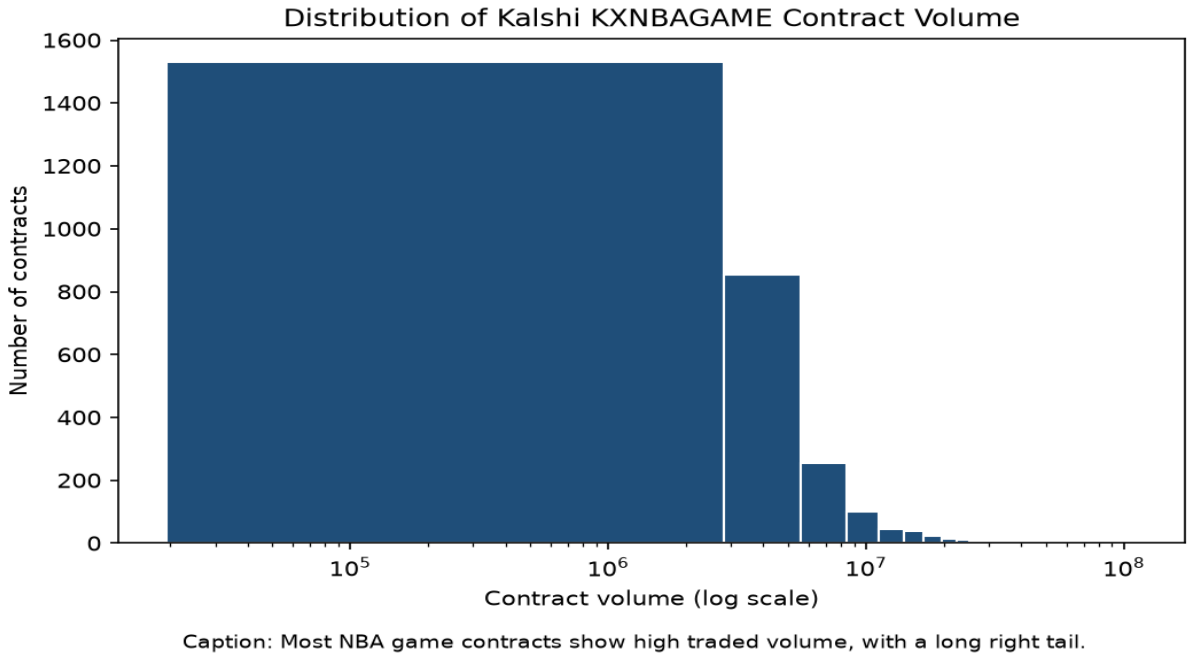
last_price_dollars: mean=0.5, std=0.4893, min=0.01, Q1=0.01, median=0.49, Q3=0.99, max=0.99 (n=2898)

settlement_value_dollars: mean=0.5, std=0.4996, min=0, Q1=0, median=0.5, Q3=1, max=1 (n=2898)

status: finalized: 2898

result: no: 1446; yes: 1446; scalar: 6

title_format: at: 1570; vs: 1328




Caption: Most NBA game contracts show high traded volume, with a long right tail.

*Figure 1. Histogram of Kalshi contract volume (log scale).*

Figure 1 shows volume_fp across Yes-contracts. A log scale is used because volume spans several orders of magnitude. Readers should take away that liquidity is generally high but uneven—useful later when conditioning calibration error on volume quartiles.

## Basketball Reference results

After de-duplication, the schedule table has 1322 rows and 7 columns covering 2025-10-21 to 2026-06-13. Rows represent One NBA game (regular season or playoffs) with final score. Columns represent Game date, visitor/home teams, and final points.

No missing values remain after de-duplication (verified with results.isna().any().any() == False).

Variables of interest: home_pts, visitor_pts, and derived point_margin / home_win. These define the modeling target (home_win) and support rest/form features built from prior games for RQ2–RQ4.

home_pts: mean=116, std=12.98, min=77, Q1=107, median=116, Q3=125, max=157 (n=1322)

visitor_pts: mean=114.2, std=12.96, min=66, Q1=105, median=114, Q3=123, max=157 (n=1322)

```
point_margin: mean=1.778, std=16.47, min=-55, Q1=-9, median=3, Q3=13, max=54
(n=1322)

home_win: 1: 734; 0: 588

home_team: San Antonio Spurs: 52; Oklahoma City Thunder: 50; New York Knicks: 50;
Cleveland Cavaliers: 50; Detroit Pistons: 49; Philadelphia 76ers: 47; Minnesota
Timberwolves: 47; Los Angeles Lakers: 46; Orlando Magic: 46; Boston Celtics: 45
```

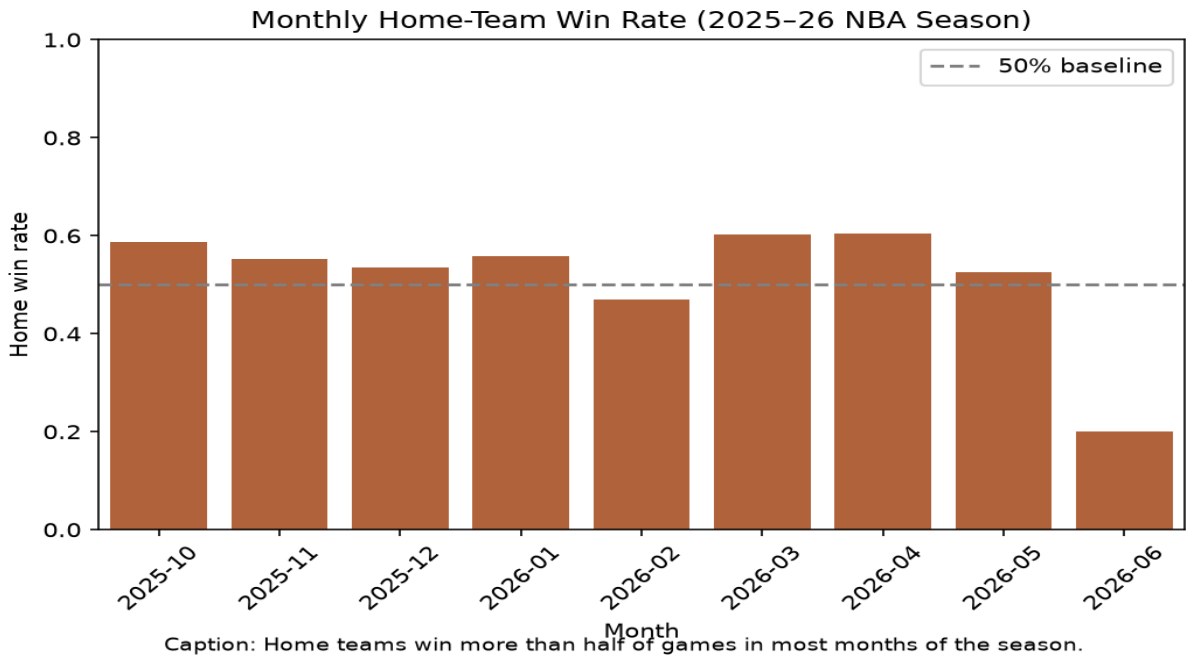



*Figure 2. Monthly home-team win rate in the 2025–26 Basketball Reference schedule.*

Figure 2 summarizes the binary home_win target over time. A bar chart by month is appropriate for a seasonal rate. The takeaway is that home-court advantage is present but not constant—motivation for including a home indicator and checking whether Kalshi prices already absorb it.

## Joined Kalshi–Basketball Reference dataset

The joined analysis table has 1316 rows and 12 columns. Rows represent One NBA game present in both Kalshi and Basketball Reference. Match rate versus Kalshi game events is 90.9%; versus BR games 99.5%. Unmatched Kalshi events include non-NBA exhibitions (e.g., Guangzhou) and any remaining label mismatches.

The joined modeling columns checked here have no missing values (isna().any().any() is False on the retained frame).

Variables of interest on the join: total_volume (RQ3 liquidity), home_win (RQ1–RQ4 outcome), home_pts/visitor_pts (feature construction / margin diagnostics), and title_format (merge-quality monitor). These are the minimum fields needed to study calibration and volume-conditioned accuracy before model training.

```
total_volume: mean=8.57e+06, std=1.12e+07, min=2.315e+05, Q1=3.567e+06,
median=5.902e+06, Q3=9.423e+06, max=1.716e+08 (n=1316)

home_pts: mean=116, std=12.96, min=77, Q1=107, median=116, Q3=125, max=157 (n=1316)

visitor_pts: mean=114.2, std=12.97, min=66, Q1=105, median=114, Q3=123, max=157
(n=1316)

home_win: mean=0.5555, std=0.4971, min=0, Q1=0, median=1, Q3=1, max=1 (n=1316)

home_win: 1: 731; 0: 585

title_format: at: 781; vs: 535
```

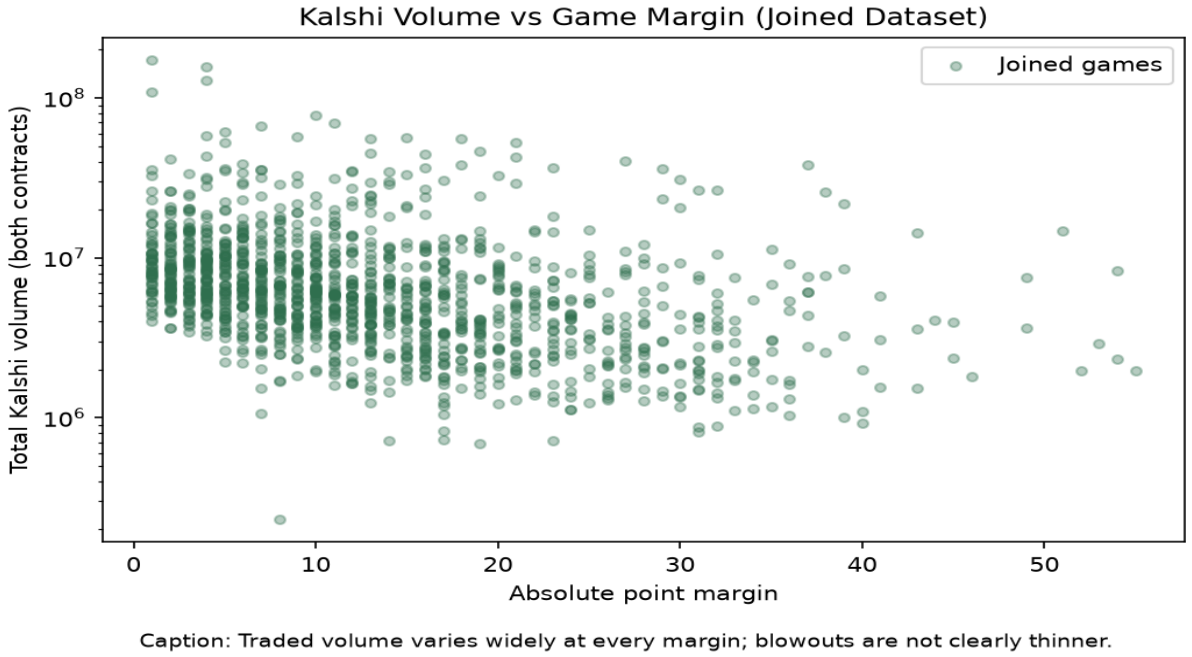



*Figure 3. Scatterplot of total Kalshi volume versus absolute point margin.*

Figure 3 jointly displays market participation and game competitiveness. A scatterplot is used because both variables are continuous. The visual suggests volume is not a simple function of final margin, so volume-based conditioning in RQ3 is not just a proxy for blowouts.

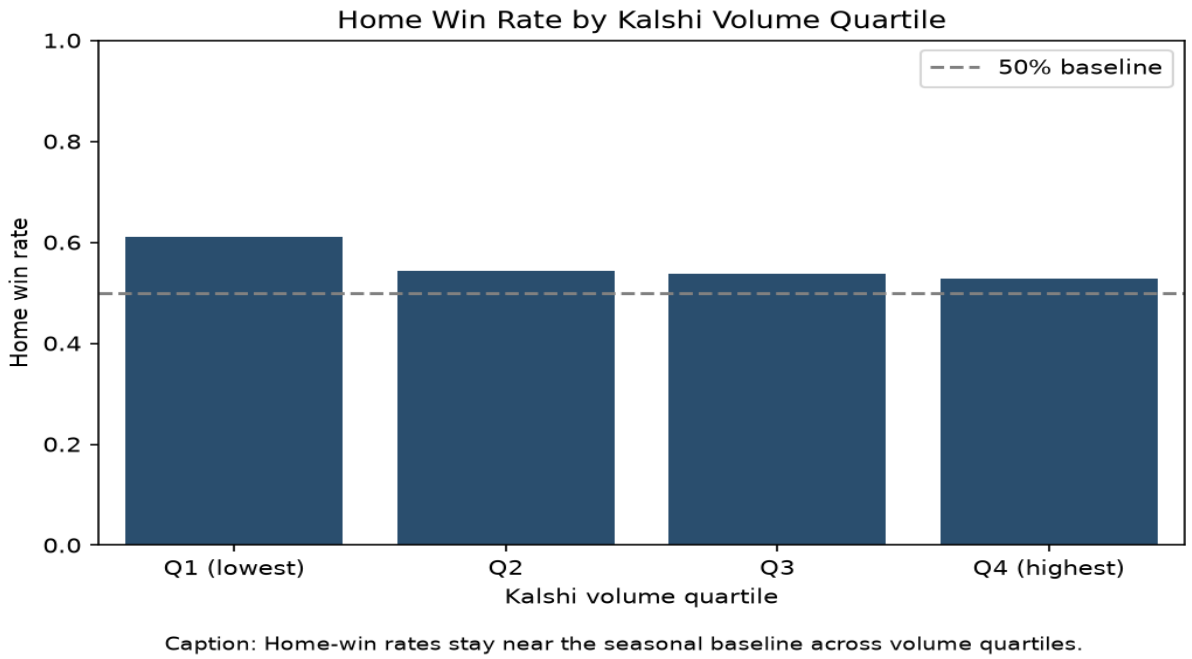



*Figure 4. Home win rate by Kalshi total-volume quartile on joined games.*

Figure 4 is an early look at RQ3-style conditioning: does outcome frequency shift with liquidity? Bars by quartile communicate group rates clearly. Rates remain near the seasonal home-win baseline, so later work should compare Brier/calibration—not only win rate—across the same groups.

## Challenge Goals Update

Messy Data (retained): Implemented pagination across Kalshi recent/historical endpoints, nested JSON flattening, title/ticker parsing for “at” and “vs” formats, timestamp fallbacks when occurrence_datetime is missing, Basketball Reference HTML table scraping with duplicate-row removal, and canonical team-name normalization for join keys. Candlestick pulls for true pregame midpoints are wired in kalshi_nba.eda.pregame and will be scaled up for modeling.

Machine Learning (unchanged plan): Still targeting logistic regression, random forest, and gradient boosting with validation-set tuning and one chronological test evaluation against Kalshi. No models are trained in this EDA deliverable; feature/split utilities already exist in the repository scaffold.

## Work Plan Evaluation

The proposal estimated ~3 hours to download/inspect and ~5 hours to clean/merge. Those estimates were somewhat optimistic for Kalshi’s split live/historical candlestick APIs and inconsistent title formats (“at” vs “vs”, truncated city labels). Collecting markets/results was faster than expected (~minutes once scripts existed), but parsing/join QA took longer than the raw download step. Feature engineering (~5h), modeling (~6h), and final visualizations/report polish (~5h) remain. Updated remaining effort is concentrated on bulk pregame candlestick collection, leakage-safe features, model

tuning, and final report figures—not on basic schedule scraping.

## Testing

EDA code is tested with pytest using small fixture files under data/fixtures and assert-style unit tests in tests/test_eda.py and tests/test_kalshi_parse.py. Tests check title/ticker parsing, seven-number summaries, missingness detection via isna(), game-level collapse to two teams per event, and successful inner joins on synthetic Kalshi+BR frames. Visual outputs are smoke-checked by running scripts/run_eda.py end-to-end on the saved raw pull. These tests support trusting reported counts and join rates because the same helper functions produce both the fixtures' expected values and the full-data EDA payload.

## Collaboration

No collaborators beyond course staff/team context. Resources consulted: Kalshi public API documentation (markets, historical cutoff/candlesticks), Basketball Reference schedule pages, pandas/matplotlib documentation, and the course CSE 163 code quality guide / flake8 expectations.

## Appendix: EDA numeric payload excerpt

The machine-readable payload written by scripts/run_eda.py is summarized above; full JSON is saved under data/processed/eda_summary.json for reproducibility.

| Dataset | Rows | Cols | Has missing |
|---|---|---|---|
| Kalshi markets | 2898 | 54 | True |
| Basketball Reference | 1322 | 7 | False |
| Joined games | 1316 | 12 | False |